BL0939 Inflated Energy — Diagnostic Findings

SuperSmart / Swigo-Gen2 firmware — power sensing path
Reference file: `src/power_sensor_bl0939.cpp`

1. Symptom

- One specific device with a BL0939 chip reports **correct voltage and current**, but **energy is inflated by roughly 300×**.
- Observed: motor running ~1 min reported ~3 kWh instead of ~0.01 kWh.
- Observed: motor running ~15 min reported ~50 kWh instead of ~0.15 kWh.
- This is a **linear, repeatable scaling** — not random spikes.
- An earlier identical case was resolved by replacing the BL0939 chip.

2. Why V and I are correct but energy is wrong

The BL0939 has **two independent measurement paths** internally:

Path	Registers used	Maps to (firmware)
RMS path	V_RMS, I_RMS	`voltage`, `current`
Active-power / energy path	WATT, CF_CNT	`power`, `energy`

Each path has its **own internal trim / calibration registers** (WA_GAIN, PHASE_OS, WA_OS, etc.). A defect in the active-power path leaves the RMS path completely unaffected — exactly the symptom seen.

3. Ruling out a sign / firmware bug

A sign-extension or bit-flip issue would produce:

- A fixed 1× / 2× error, or
- Random huge spikes when the high bit toggles.

It would **not** produce a clean ~300× scaling that grows linearly with run time. So the firmware sign handling is **not** the cause of this particular failure.

> **Side note (worth fixing anyway):** in `Decode()` the variable `energyATemp` is declared `int32_t` but assigned 24 raw bits and never sign-extended (unlike `powerATemp`). Per the BL0939 datasheet CF_CNT is signed 24-bit. This will not cause a 300× inflation, but it can cause occasional very large positive jumps and is worth tightening up.

4. Most likely root cause

A **factory trim error on this specific chip die**.

What "die trim" means

- Inside the chip there are small analog variations from one piece of silicon to the next.
- At end of manufacture, the factory tester writes correction constants into **on-die OTP / fuses**.
- The chip reads those constants at every power-up and uses them to scale its measurements.
- If that calibration step recorded the wrong values for this die, the chip wakes up with a bad gain baked in — **permanently and consistently**.

This is **not mechanical damage**. The chip is electrically intact; it is simply calibrated wrong from the factory. The defect is invisible until energy is measured under load.

Which trim is wrong

Suspect	Plausibility for ~300×
WA_GAIN (active power gain)	High — direct multiplicative error on WATT and therefore CF_CNT.
CF pulse-weight trim	High — each counted pulse represents far more energy than firmware assumes.
PHASE_OS (phase offset)	Low — caps at roughly 2–3×, cannot explain 300×.

5. Why chip replacement worked previously

Replacing the BL0939 swaps in a die that was trimmed correctly. Nothing about the firmware, PCB, or surrounding circuit changed. This confirms the defect is contained inside the chip's OTP / trim values.

6. Remediation options

A. Continue replacing chips

Acceptable if occurrences are rare. No code change. Cost: RMA per unit.

B. Add a per-device energy correction factor (recommended)

Expose a server attribute (similar to existing `I_correction_factor_*`) that scales energy in firmware. A badly trimmed but otherwise functional chip can then be rescued by setting the attribute remotely, with no return needed.

```
energy = ((float)energyATemp * CF_Conversion_Shunt)
         / E_correction_factor_SHUNT
```

```
        / deviceEnergyCorrection;   // default 1.0, set per device from cloud
```

Same pattern can be applied to `power` if that telemetry is also published.

C. Explicitly initialise WATT-path trim registers in BL0939_setup()

Today, only RMS-related registers are written during setup (`REG_MODE`, `REG_TPS_CTRL`, `REG_IA/IB_FAST_RMS_CTRL`). The WATT-path trim registers (WA_GAIN, PHASE_OS_A, WA_OS, and B siblings) are left at whatever the chip booted with. Writing known values gives a deterministic starting point across chips and may mask some bad-trim cases.

D. Derive energy from `power × Δt` in firmware

Bypasses CF_CNT entirely. Helps only if CF_CNT itself is the broken part; does not help if the underlying WATT calculation is wrong (which appears to be the case here).

E. Fix the sign-extension of CF_CNT (low-risk hygiene)

Apply the same 24-bit sign extension already used for `powerATemp`. Does not fix the 300× case, but eliminates a separate failure mode.

```
if (bitRead(energyATemp, 23)) energyATemp |= 0xFF000000;
```

7. Recommendation

> Implement **option B** (per-device energy correction factor) plus **option E** (sign-extension hygiene). Together they convert a hardware RMA into a remote configuration change for inflated-energy cases, and close off the unrelated sign-bit failure mode at minimal risk.

Prepared for: info@eviqtech.com | Date: 2026-05-12